实验 4 函数

一. 实验目的

理解函数的作用，体会函数是一种简化程序结构的重要手段。掌握函数的定义和使用方法。理解调用函数与被调函数之间的数据传递。理解函数原型的作用，会在程序中使用。掌握数组作为函数参数传递的使用方法。能用函数解决实际问题。掌握函数的嵌套调用和递归调用，会使用递归的思想解决问题。能够理解函数调用过程中形式参数和实际参数的关系。理解存储类型和生命周期，弄清楚主函数和自定义函数中的变量存储和生存。。

二. 知识要点

1. 函数的定义与使用

函数（function）是完成特定任务的独立程序代码单元。程序设计中为什么要使用函数？一方面，使用函数可以实现代码的重用，隐藏实现的细节。如果程序要多次完成某项任务，那么只需编写一个合适的函数，就可以在需要时使用这个函数，或者在不同的程序中使用该函数。同时，对于函数的调用者来说，只需要关注函数的接口，不需要了解实现细节。另一方面，从程序模块化考虑，会把任务分解成函数来实现，即使这项任务只用执行一次，从而提高程序代码的可读性，也更方便后期的代码维护和升级。

如果只是使用函数，只需要关心其需要的输入信息、产生的响应或输出值。否则需要知道如何正确地定义函数、如何调用函数和如何建立函数间的通信。

函数和变量一样，有多种类型。任何程序在使用函数之前都要声明该函数的类型。一般而言，函数原型指明了函数的返回值类型和函数接受的参数类型。在实际应用中，要学会区分函数原型、函数调用和函数定义。函数原型（function prototype）告诉编译器函数的类型；函数调用（function call）表明在此处执行函数；函数定义（function definition）明确地指定了函数要做什么，提供实际的代码。以下面代码为例，第一次出现的 square 是函数原型，第二次出现的是函数调用，第三次出现的是函数定义。

```
#include <iostream>
using namespace std;
int square(int x);      //函数原型
int main(void)
{
   int x;
   for (x=1; x<=10; x++)
      cout<<square(x)<<"    ";     //函数调用
   return 0;
}
int square(int x)      //函数定义
{
   return x*x;
}
```

2. 函数的参数和返回值

前面代码定义 square(int x)函数中的 x 是 int 类型，这个变量被称为形式参数，简称形参。它和定义在函数中的变量一样都是局部变量。每次调用函数，就会给形参赋值。形式参数是被调函数中的变量，实际参数（简称实参）是主调函数赋给被调函数的具体值。如上面代码的主函数中调用 square(x)时的参数 x 是实参，在循环中依次等于值 1~10，这些值在每次调用函数时被拷贝给被调函数的对应形参 x。值得注意的是，这里的形参和实参虽然都用标识符 x 表示，但代表了不同的变量。

在 C++语言中，所有调用都是按值调用，也就是把参数的值的副本传递给被调函数，在被调函数内改变参数的值不会影响主调函数中的参数的原始值。

声明函数时必须声明函数的类型。带返回值的函数类型应该与其返回值类型相同，而没有返回值的函数应声明为 void 类型。

3. 函数的嵌套调用

在 C++语言中，函数间的关系是平行的、独立的，不允许嵌套定义，但允许嵌套调用，即在调用某函数过程中又调用另一函数。

以计算圆环的面积为例理解函数的嵌套调用。圆环的面积可以由外圆的面积减去内圆的面积获得，因此可以定义两个函数，函数 circleArea 计算圆的面积，函数 ringArea 计算圆环的面积。函数 ringArea 调用 circleArea 分别计算外圆和内圆的面积后做减法得到圆环面积，这就是函数的嵌套调用。代码如下：

```
#include <iostream>
using namespace std;
#define PI 3.14
double circleArea(double r);
double ringArea(double r1, double r2);
int main(void) {
   double r1, r2, s;
   cout<<"\tInput r1, r2: ";
   cin>>r1>>r2;
   s = ringArea(r1, r2);
   cout<<"\n\tThe area is:"<<s<<"\n";
}
double circleArea(double r)
{
      return PI*r*r;
}
double ringArea(double r1, double r2)
{
   if (r1<=r2)
      return circleArea(r2)-circleArea(r1);
   else
```

```
    return circleArea(r1)-circleArea(r2);
}
```

4.函数的递归调用

在函数调用过程中，直接或间接调用自己的函数称为递归函数。递归函数必须包含能让递归调用停止的语句。通常，递归函数都使用 if 或其他等价的测试条件在**函数形参等于某特定值时终止递归**。为此，每次递归调用的形参都要使用不同的值。

最简单的递归形式是把递归调用置于函数的末尾，即正好在 return 语句之前。这种形式的递归被称为尾递归，它相当于循环。

以计算阶乘为例，5! = 5 * 4 * 3 * 2 * 1=5 * 4!，4! = 4 * 3!，……，可以表示为公式 n != n*(n-1)! ，基本情形是 n=0 或 n=1 时，阶乘为 1。代码如下：

```
#include <iostream>
using namespace std;
long fac(int n)
{
    long f;
    if (n < 0)
        return -1;
    if (n == 0 || n == 1)
        f = 1;
    else
        f = n * fac(n - 1);
    cout<<"\t"<<n<<"!="<<f<<"\n";
    return f;
```

```
}
int main(void)
{
    cout<<"\n\t5!="<<fac(5)<<endl;
}
```

用递归解决问题时，递归函数只知道如何去解最简单的情形，如简单的返回一个值。这时可以把复杂的问题分成两部分，一部分是函数知道如何去做的部分，另一部分是函数不知道如何去做的部分。而后一部分与原问题相似，且更简单，函数可以调用自己的新形式来解决这个更小的问题，这就是递归调用。当调用最终遇到基本情形时，函数识别出状态并将结果返回给前一个情形，这样，一系列的结果就按顺序返回，直到把最终结果返回给原始的调用者。

5. 存储类型和生存期

与存储类别相关的主要关键字有 auto、extern、static 和 register。不同的存储类别具有不同的存储期、作用域和链接。

作用域描述程序中可访问标识符的区域。一个 C 变量的作用域可以是块作用域、函数作用域、函数原型作用域或文件作用域。旧的标准要求具有块作用域的变量都必须声明在块的开头。C99 标准放宽了这一限制，允许在块中的任意位置声明变量。

作用域和链接描述了标识符的可见性。存储期描述了通过这些标识符访问的对象的生存期。C 对象有 4 种存储期：静态存储期、线程存储期（不做讨论）、自动存储期、动态分配存储期。变量具有静态存储期，意味着它在程序的执行期间一直存在。文件作用域变量具有静态存储期。块作用域的变量通常都具有自动存储期。当程序进入定义这些变量的块时，为这些变量分配内存；当退出这个块时，释放刚才为变量分配的内存。块作用域变量也能具有静态存储期，这需要在变量声明前加上关键字 static，这样定义的变量从程序被载入到程序结束期间都存在，但是作用域是在定义它的函数块中。malloc()函数可以在程序运行时分配

内存，这部分内容在指针部分讨论。

变量，其实是内存地址的一个抽像名字。在静态编译的程序中，所有的变量名都会在编译时被转成内存地址。机器是不知道我们取的名字的，只知道地址。内存的使用是程序设计中需要考虑的重要因素之一，不仅由于系统内存是有限的（尤其在嵌入式系统中），而且内存分配也会直接影响到程序的效率。在 C 语言中，定义了 4 个内存区间：代码区、全局变量和静态变量区、局部变量区即栈区、动态存储区即堆区。栈区 stack 由编译器自动分配释放，存放函数的参数值，局部变量的值等。堆区 heap 一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。全局区（静态区 static）：全局变量和静态变量的存储是放在一块的，初始化的全局变量和静态变量在一块区域，未初始化的全局变量和未初始化的静态变量在相邻的另一块区域。程序结束后由系统释放。常量区：常量字符串就是放在这里的。程序结束后由系统释放。程序代码区存放函数体的二进制代码。

静态内存的数量在编译时确定；静态数据在载入程序时被载入内存。在程序运行时，自动变量被分配或释放，所以自动变量占用的内存数量随着程序的运行会不断变化。可以把自动内存看作是可重复利用的工作区。动态分配的内存也会增加和减少，这个过程由函数调用来控制。

三. 示例讲解

示例 1. 观察下面代码中形参和实参的数值变化。

```
#include <iostream>
using namespace std;
void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
    cout<<"1st: " <<a<<","<<b<<endl;
```

```
}
int main()
{
   int a=6, b=5;
   swap(a, b);
   cout<<"2nd: "<<a<<","<<b<<endl;
}
```

由于形参只是实参的值拷贝，swap( )函数只对形参进行了交换，而对实参没有任何影响。

示例 3. 把数组作为参数传递给函数,注意参数的写法。

```
#include <iostream>
using namespace std;
#define SIZE 5
void modifyArray(int array[], int size)
{
    int j;
    for (j = 0; j < size; j++)
        array[j] *= 2;
}
void modifyElement(int e)
{
    cout<<"\nValue in modifyElement is "<< (e*=2);
}
int main(void)
{
    int a[] = {0, 1, 2, 3, 4}, i;
    cout<<"\nThe values of the original array are:\n";
    for (i = 0; i < SIZE; i++)
        cout<<" "<<a[i];
```

```
    modifyArray(a, 5);
    cout<<"\nThe values of the modified array are:\n";
    for (i = 0; i < 5; i++)
        cout<<" "<<a[i];
    cout<<"\nThe value of a[3] is "<<a[3];
    modifyElement(a[3]);
    cout<<"\nThe value of a[3] is "<<a[3];
    return 0;
}
```

上面代码的运行结果是：

The values of the original array are:

0 1 2 3 4

The values of the modified array are:

0 2 4 6 8

The value of a[3] is 6

Value in modifyElement is 12

The value of a[3] is 6

函数 modifyArray(int array[], int size)用数组名 array 做参数，数组名做参数时，通常要把数组的长度传递给函数，参数 size 代表的是形参数组长度。数组名是数组第一个元素的地址，按值传递所以主调函数传递的是地址值，那么在形参中的变量 array 被赋值为地址值，则形参数组与实参数组存储地址一致，函数对形参数组值的操作也会改变实参数组的值。而函数 modifyElement(int e)则不会影响实参。

四．实验内容（这次练习不能只用 main 函数完成。完成后将程序与运行结果同屏截图，放在邮件里发送本次上机作业）

1．编程实现函数 $f(x)=(x+1)^2$ 和 $g(x)=2x+1$ 的功能，然后在程序主函数中输入 x 值并调用函数输出 f(g(x))的值。注意体会函数的嵌套调用。

2. 编程实现下述功能：主函数接收用户输入的 3-6 之间的一个正整数，然后调用自定义的函数 Draw 输出由*组成的正方形。如用户输入 4，则输出下面图形。函数原型是：void Draw(int n)。

```
* * * *
*     *
*     *
* * * *
```

3. **使用函数**编程实现一个金秋服饰大赛**打分系统**，完成下列功能：

某学院表演完后，十个评委输入分数(0-10 分)后，利用函数 Sort 将分数从大到小排序，再利用函数 Ave 计算去掉一个最高分和一个最低分后的平均分，在主函数中输出平均分并从大到小输出分数。主函数中再增加两个评委打分，再次调用 Sort 和 Ave 函数完成新平均分和 12 个分数输出。

函数原型：void Sort(float a[], int n);　　float Ave(float a[], int n);

4. **换钱工具。**主函数中接受用户输入一个指定的金额（以元为单位，如 345），然后调用函数完成换钱工作，最后在主函数中输出换钱方案，显示支付该金额的各种面额人民币数量。要求显示 100 元、50 元、10 元、5 元、2 元、1 元各多少张。要求：先按大面额换算，如输出：345=100*3+10*4+5*1。（函数间要交互多个数据，想想如何设计函数参数。）

5. 编写一个函数 Check 用于检查一个整数能不能表示成两个质数之和，并返回判断结果（是或否）。要求：在主函数中接收用户输入的数，调用函数 Check 判断后，在主函数中输出判断结果。

函数原型：int Check(int number);　　int Zhishu(int number);

6. **杰克与艾米丽。**

夜幕低垂，星光点缀着深邃的天幕，一场盛大的舞会在华丽的宴会厅中缓缓落下帷幕。音乐声渐渐远去，留下的是一地的欢声笑语和那些在舞池中旋转的身影。

程序员杰克站在角落，目光穿过人群，定格在一位穿着蓝色长裙的女孩身上。她的笑容如同春日的暖阳，温暖而明媚，让他整晚都难以移开视线。女孩名叫艾米丽，她的眼睛里仿佛藏着星辰，每一次眨眼都像是在诉说着一个神秘的故事。杰克知道，他必须在今晚结束之前，鼓起勇气去要她的联系方式。舞会的灯光渐渐暗淡，宾客们开始三三两两地离开。杰克

深吸一口气，调整了一下领带，迈开了步伐。他的心跳在胸腔中如同鼓点般急促，每一步都像是在跨越一道道无形的障碍。艾米丽正和朋友们谈笑着，她没有注意到杰克的接近。直到他站在她的面前，她才抬起头，那双明亮的眼睛里闪过一丝惊讶。

“嗨，艾米丽，我是杰克。今晚的舞会真是太精彩了，尤其是和你一起跳舞的时候。”他的声音有些颤抖，但还算镇定。

艾米丽微微一笑，她记得这个在舞池中与她共舞的男孩，他的舞步虽然有些生疏，但真诚而热情。“谢谢你，杰克。我也很高兴能和你跳舞。”

杰克鼓起勇气，继续说道：“我不知道这么说会不会太唐突，但是……我希望能再次见到你。不知道你愿不愿意给我你的联系方式？”

艾米丽的眼中闪过一丝玩味的光芒，她喜欢这个男孩的直率和勇气。“当然可以，杰克。我也希望能和你保持联系。”

她从手包中拿出一支笔，在杰克的手掌上写下了一串数字。杰克感到一阵温暖从她的指尖传来，他的心中充满了喜悦和期待。

“将我手机号每一位上的数字加 5 再对 10 求余后作为新的一串数字，然后将其右移 3 位，再倒序，就是你手里的数字。”艾米丽说完和朋友离开了。

随着宾客们的离去，舞会的喧嚣渐渐平息。杰克握着手中的号码，心中充满了对未来的憧憬，他打开掌心，读了出来：23105611981。

杰克回到家，打开电脑，开始编程，屏幕的光打在他微红的脸上。。。

他在主函数中输入加密后的手机号字符串，调用函数 Jiemi 完成解密，然后在主函数中输出了手机号。

函数原型：void Jiemi(char input[], char output[]);

7. 信号滤波器。均值滤波器和中值滤波器是信号处理中十分常用的噪声去除方法，请阅读下面的说明后完成平滑滤波和中值滤波后输出各自的结果数据。

* 窗口宽度为 3 的均值平滑滤波：$x_i' = \frac{x_{i-1} + x_i + x_{i+1}}{3}$。两端的数据可以保持不变。

* 窗口宽度为 3 的中值滤波：$x_i' = median(x_{i-1}, x_i, x_{i+1})$，median 表示三个值的中间值（不是最大、也不是最小值）。两端的数据可以保持不变。

要求：主函数中输入原始数据，分别调用均值滤波函数和中值滤波函数，在主函数中输出两者的处理结果。

函数原型：void AveFilter(float a[], int n);　　void MedFilter(float a[], int n);

8. 编程用递归函数计算一个数组所有元素的积，并写主函数验证。

9. 编写一个递归函数，将一个整数 n 转换成字符串（例如输入整数 4679，应输出字符串“4679”），并写主函数验证。

10. 编写一个函数（功能自定义），当该函数被主函数调用 10 次时输出信息“已被调用了 10 次！”。请使用全局变量和静态局部变量两种方式完成。

11. 数组合并。主函数中用户输入两个有序数组，调用函数 Merge 合并成一个有序数组，最后在主函数中输出新数组。

函数原型：void Merge(int a[], int m, int b[], int n, int c[]);

12. **汉明距离。**编程实现如下过程：将两个位数相同的正整数转换为字符串，然后求得两个字符串之间的汉明距离 (汉明距离即相同位置而字符不同的个数)。比如将正整数 1024 和 2048 转换为字符串 "1024" 和 "2048"，它们的汉明距离为 3。编程要求：

（1）编写函数 int IntToStr(int n, char str[])，实现将正整数 n 转换为字符串 str，并返回 n 的位数；

（2）编写函数 int differ(char str1[],char str2[])，返回字符串 str1 和字符串 str2 的汉明距离；

（3）编写主函数实现从键盘输入两个正整数，调用 IntToStr 函数将它们转换为字符串，然后调用 differ 函数求得它们的汉明距离，在主函数中输出。

13. **进制转换。（选做）**编写四个函数分别实现十进制转换成十六进制，十六进制转换成十进制，十进制转换成八进制，八进制转换成十进制。然后在主函数中，用户可以选择从几进制转到几进制，然后输入数据，调用上述函数后，在主函数中输出转换结果。例如：用户选择十进制转十六进制，然后输入 255，屏幕输出 FF

：